

Mental Math Progression & Level System (v1)

Christian Köhler

1 Introduction

1.1 Long-Term Vision

The long-term vision for the mental math application is to provide a highly personalized, adaptive training experience. The system is structured around independent mathematical domains, each featuring a progressive ladder of calculation patterns.

- **Level-based training:** Each level corresponds to a specific pattern of calculations to master.
- **Domain-specific ladders:** User progress is tracked independently per domain, meaning each domain has its own level ladder.
- **Personalized progression:** Advancement is tailored to the user and evaluated based on:
 - *Correctness:* Whether the calculation was solved correctly.
 - *Speed:* How fast the calculation was solved compared to other users.
- **Comprehensive domains:** The full system will eventually support a wide range of operations, including Addition, Subtraction, Multiplication, Division, Root, Square, and Power.
- **Adjustable difficulty:** Users can manually tune their difficulty level (e.g., warm up, normal, hard).
- **Customizability:** Training configurations will include toggles to allow for negative numbers (minus) and decimals.

1.2 Version 1 Scope & Constraints

Version 1 establishes the core mechanics. It is a simplified implementation of the long-term vision with the following constraints:

- **Limited domains:** Only Addition (ADD), Subtraction (SUB), Multiplication (MUL), and Division (DIV) are available.
- **Fixed difficulty parameters:** Difficulty settings (warm up, normal, hard) are disabled. Time cutoffs for correct solves are hard-coded rather than percentile-based.
- **No customizability:** Calculations exclusively feature positive numbers and no decimals.

2 Mechanics for Progression & Level System

2.1 Pattern-Based Levels

Each domain is divided into ten progressive levels. A **pattern** specifies the constraints for the questions generated at that level.

Generation Rules

To ensure an appropriate and consistent learning curve, all patterns adhere to the following global rules:

- **Positive Results Only:** Subtraction questions never yield negative numbers.
- **Exact Division:** All division questions result in exact integers, with no remainders.
- **Meaningful Divisions:** Both divisors and quotients are always ≥ 2 to prevent trivial cases (e.g., $n \div 1$ or $n \div n$).
- **Meaningful Multiplications:** Multiplication operands are always ≥ 2 to prevent trivial cases (e.g., $n \times 1$).
- **Operand Order Randomization:** For commutative operations (Addition and Multiplication) with differing digit lengths, the display order is randomized (e.g., a 1-digit and 2-digit addition can appear as $17 + 2$ or $2 + 17$).

Progression Tables

The domain ladders are defined in the tables below based on the following metrics:

- **Level** — Sequential difficulty step within each domain (1 = easiest).
- **Operands** — The number of digits per operand (e.g., 2×1 means a 2-digit number and a 1-digit number).
- **Complexity (Carries / Borrows / Quotient)** — The operations required to solve the calculation. For Addition, Subtraction, and Multiplication, this refers to the exact number of carries or borrows. For Division, this indicates the number of digits in the resulting **quotient**.
- **Cutoff (ms)** — Minimum time budget per question to achieve a “solved” status in v1.

Addition (ADD)

Level	Operands	Carries	Cutoff (ms)
1	$1 + 1$	0	4 000
2	$1 + 1$	1	4 000
3	$1 + 2$	0	4 500
4	$1 + 2$	1	4 500
5	$2 + 2$	0	5 000
6	$2 + 2$	1	5 000
7	$2 + 2$	2	5 500
8	$2 + 3$	0	5 500
9	$2 + 3$	1	6 000
10	$3 + 3$	0	6 000

Subtraction (SUB)

Level	Operands	Borrows	Cutoff (ms)
1	$1 - 1$	0	4 000
2	$2 - 1$	0	4 500
3	$2 - 1$	1	4 500
4	$2 - 2$	0	5 000
5	$2 - 2$	1	5 000
6	$3 - 1$	0	5 000
7	$3 - 1$	1	5 500
8	$3 - 2$	0	5 500
9	$3 - 2$	1	6 000
10	$3 - 3$	0	6 000

Multiplication (MUL)

Level	Operands	Carries	Cutoff (ms)
1	1×1	0	4 000
2	1×1	1	4 000
3	1×2	0	4 500
4	1×2	1	4 500
5	1×2	2	5 000
6	1×3	0	5 000
7	1×3	1	5 500
8	1×3	2	5 500
9	1×3	3	6 000
10	2×2	0	6 000

Division (DIV)

Level	Operands	Quotient	Cutoff (ms)
1	$1 \div 1$	1	4 000
2	$2 \div 1$	1	4 000
3	$2 \div 1$	2	4 500
4	$2 \div 2$	1	5 000
5	$3 \div 1$	2	5 500
6	$3 \div 1$	3	6 000
7	$3 \div 2$	1	6 500
8	$3 \div 2$	2	7 000
9	$4 \div 2$	2	7 500
10	$4 \div 2$	3	8 000

2.2 Progression System

User progression through the levels of a given domain is evaluated based on recent performance. An attempt is considered **solved** if the correct answer is submitted on the first try and within the level’s hard-coded cutoff time C_L . Otherwise, the attempt counts as **unsolved**.

Promotion

To advance from the current level L to $L + 1$, the user must demonstrate consistent mastery. The system evaluates the last $N = 5$ completed attempts at Level L . Promotion requires that **all** of the last N attempts are solved. Once this condition is satisfied, Level $L + 1$ is unlocked.

Demotion

To prevent users from being stuck on a level that is too difficult, a demotion rule is enforced based on the last $M = 10$ attempts at the current level. A user is demoted if the unsolved rate over these M attempts exceeds a maximum tolerance ($S_{max} = 0.25$ or 25%). Demotion is only allowed after a minimum exposure of M attempts at that level.

Soft Demotion: Demotion is intentionally soft. If the user is at level L , a demotion moves them to $L - 1$ (with a minimum level of 1). This applies even if the triggering failures happened on a sampled lower-level question $k < L$. For example, if a user at level 10 repeatedly fails sampled level 8 questions, their new level is 9. Formally:

$$L_{new} = \max(1, L - 1)$$

2.3 Level Selection Algorithm

To balance current level mastery with the review of previous levels and introduction to the next level, the system dynamically samples question levels during play. The sampling algorithm heavily weights towards the user’s current level, assigns a fixed probability to the next level, and uses a geometrically decaying probability for all lower levels.

Symbol Definitions

- L : current user level in the domain
- k : sampled question level ($k \in \{1, 2, \dots, L, L + 1\}$)
- r : geometric decay factor for lower levels ($r \in (0, 1)$). Smaller r means faster taper.

- $u(k)$: unnormalized geometric weight for lower levels

Sampling Formula

The base probability mass is statically distributed to the upper levels:

$$p(L) = 0.50, \quad p(L+1) = 0.10$$

The remaining 0.40 mass is allocated to all lower levels (1 to $L-1$) using a geometric taper so that levels closer to L are more probable:

$$u(k) = r^{(L-1)-k}, \quad 1 \leq k \leq L-1$$

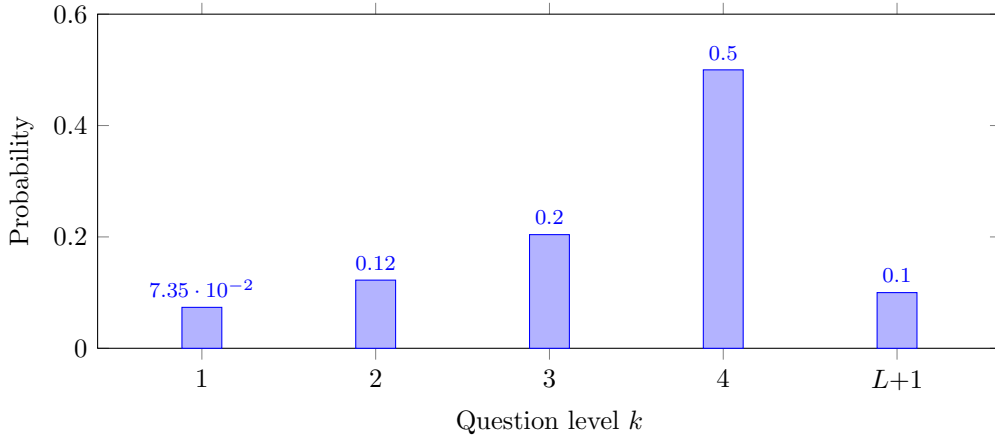
$$p(k) = 0.40 \cdot \frac{u(k)}{\sum_{j=1}^{L-1} u(j)}, \quad 1 \leq k \leq L-1$$

Edge Case ($L = 1$): The distribution simplifies to:

$$p(1) = 0.90, \quad p(2) = 0.10$$

Example Distribution

Assuming a user is at level $L = 4$ with a decay factor $r = 0.6$, the resulting probabilities $p(k)$ taper off logarithmically for the lower levels:



3 Data Architecture

3.1 Question Generation & Storage

The system does not hard-code individual questions; rather, it stores generation patterns and derives questions on the fly:

1. Store **Pattern** rows (the generation rules).
2. Generate question instances on demand using a Random Number Generator (RNG) seed.
3. Store each user interaction as an **Attempt** row.

In practice, a “question” is represented purely by its identifier pair: (**patternId**, **seed**).

3.2 Database Schema

Each pattern family corresponds to a row in the `pattern` table. Stable, indexed fields are stored as standard columns:

- `id`: unique pattern identifier
- `domain`: defined as `ADD | MUL | SUB | DIV`
- `level`: ladder level within the domain
- `description`: human-readable pattern label
- `cutoffTimeMs`: solved-time cutoff for v1
- `active`: boolean to enable/disable the pattern
- `createdAt, updatedAt`: audit timestamps

In v1, the uniqueness constraint is defined as: `UNIQUE(domain, level, description)`. Dynamic generation constraints are stored in a secondary `params` column (using Prisma `Json` or PostgreSQL `JSONB`).

3.3 Parameter JSON Structures

In v1, digit fields within `params` are stored as single integers (rather than lists) because each pattern uses one fixed digit size per side.

Addition (ADD) Example

Level 3: 1-digit + 2-digit, no carry

```
{
  "operation": "ADD",
  "leftDigits": 1,
  "rightDigits": 2,
  "carryCount": 0
}
```

Subtraction (SUB) Example

Level 5: 2-digit - 2-digit, single borrow

```
{
  "operation": "SUB",
  "leftDigits": 2,
  "rightDigits": 2,
  "borrowCount": 1,
  "allowNegative": false
}
```

Multiplication (MUL) Example

Level 5: 1-digit x 2-digit, two carries

```
{
  "operation": "MUL",
  "leftDigits": 1,
  "rightDigits": 2,
  "carryCount": 2
}
```

Division (DIV) Example

Level 3: 2-digit dividend / 1-digit divisor, 2-digit quotient

```
{  
  "operation": "DIV",  
  "dividendDigits": 2,  
  "divisorDigits": 1,  
  "quotientDigits": 2,  
  "exactInteger": true  
}
```